KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY (KIIT)

Deemed to be University U/S 3 of UGC Act, 1956

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY (KIIT)

School of Engineering

SAP CAPSTONE PROJECT REPORT

Topic VI — Real CAP Development Scenario

Procure-to-Pay Analytics Dashboard using SAP Cloud Application Programming Model

Domain: SAP Cloud Application Programming Model (CAP)
Certification Track: SAP Certified Associate — Backend Developer (SAP CAP)
University: KIIT University
Academic Year: 2025 – 2026
Submitted To: SAP Practice Faculty, School of Engineering

Submitted By: Anula Mishra
Roll No.: 2305843

1. Introduction and Topic Justification

This report documents the design, development, and deployment of a Procure-to-Pay (P2P) Analytics Service built on the SAP Cloud Application Programming Model (CAP) as part of the SAP Backend Developer Capstone Project at KIIT University for the academic year 2024–25. The project was undertaken in partial fulfilment of the SAP Certified Associate Backend Developer SAP CAP certification track.

Among the six project topics prescribed for this course, I selected Topic VI - Real CAP Development Scenario because it most directly aligns with the skills that the backend developer certification examines. The following points explain that choice.

Alignment with Certification Objectives

The SAP CAP certification syllabus focuses on CDS data modelling, OData V4 service exposure, server-side business logic through event handlers, XSUAA-based authentication, and BTP Cloud Foundry deployment. Every one of these areas is exercised in a real development scenario. Choosing a functional topic such as P2P or O2C would have focused effort on configuration steps rather than on writing and testing actual code, which is not what the certification measures.

Backend-First Scope

This topic required me to write CDS entity definitions, compose analytical views, implement before/on/after event handlers in Node.js, define OData actions and functions, and author Jest-based unit and integration tests. This is the nature of backend developer work and it is what I wish to demonstrate through this submission.

End-to-End Completeness

By scoping the project around the full P2P business cycle from Purchase Requisition through Purchase Order, Goods Receipt, and Invoice Verification the project remains realistic and domain-relevant while staying within the reach of a single developer working over the course duration.

Industry Relevance

SAP BTP and CAP are the current strategic platform for all new SAP side-by-side and extension development. Practical experience with this stack is increasingly valued by employers over classic ABAP or configuration knowledge, making this topic the most career-relevant choice as well.

2. Fictitious Company and Organisational Setup

2.1 Company Profile

For the purpose of this project I defined a fictitious company called NovaBuild Manufacturing Pvt. Ltd. The company serves as the business context within which all procurement transactions are modelled and tested. The details below describe the SAP organisational setup I built around this entity.

Attribute	Details
Company Name	NovaBuild Manufacturing Pvt. Ltd.
Industry	Industrial Equipment Manufacturing
Headquarters	Bhubaneswar, Odisha, India
SAP System	SAP S/4HANA 2023 (On-Premise) + SAP BTP (Extension)
ERP Client	100 (Production) / 200 (Development)
Fiscal Year	April to March (Indian FY)
Currency	INR (Indian Rupee)
Company Code	NB01

2.2 Organisational Structure

The following SAP organisational entities form the backdrop for all procurement transactions handled by the CAP service. I defined these entities to mirror the kind of multi-plant, multi-purchasing-group setup that a real Indian manufacturer would use.

SAP Entity	ID	Description
Company Code	NB01	NovaBuild Manufacturing — legal entity for FI postings
Plant	NB10	Bhubaneswar Main Plant — Production and Warehousing
Plant	NB20	Pune Satellite Plant — Assembly Unit
Storage Location	SL01	Raw Material Store
Storage Location	SL02	Finished Goods Store
Purchasing Organisation	NBPO	Centralised Purchasing Organisation
Purchasing Group	PG01	Direct Materials Buyers
Purchasing Group	PG02	Indirect / MRO Buyers
Sales Organisation	NBS1	India Domestic Sales
Distribution Channel	DC	Direct Customer
Division	ME	Mechanical Equipment
Controlling Area	NB01	NovaBuild Controlling Area
Cost Centre	CC100	Production Department
Cost Centre	CC200	Finance and Administration
Profit Centre	PC001	Mechanical Equipment Profit Centre

2.3 Sample Vendor Master Data

Ten vendor records were seeded into the system to represent real Indian industrial suppliers. A representative selection is shown below. All GST and PAN numbers follow the correct Indian format and were constructed specifically for this project.

Vendor ID	Name	State	Payment Terms	Credit Limit (INR)
V001	Tata Steel Limited	Maharashtra	NT30	50,00,000
V004	Siemens India Limited	Maharashtra	NT60	80,00,000
V005	Bosch India Ltd	Karnataka	NT30	40,00,000
V007	Thermax Ltd	Maharashtra	NT60	60,00,000
V008	Larsen and Toubro Ltd	Maharashtra	NT30	1,00,00,000

3. Project Architecture

3.1 System Landscape

The project follows the standard SAP BTP multi-layer architecture recommended for CAP-based extensions. The diagram below summarises the five layers and how they relate to one another.

Layer 1 — Presentation (SAP Fiori Elements List Report, Postman, REST Client)
Layer 2 — OData V4 API (/p2p/) exposed by SAP CAP Node.js runtime
Layer 3a — CDS Data Models (db/schema.cds, db/views.cds)
Layer 3b — Business Logic Handlers (srv/p2p-service.js)
Layer 4 — SAP HANA Cloud (production) / SQLite (local development)
Layer 5 — SAP BTP Cloud Foundry runtime — deployed via mta.yaml

3.2 Technology Stack

The following technologies were used to build and deploy the project. Selections were made to align with the SAP CAP certification syllabus and to use only tools that are available on the SAP BTP free-tier trial.

Component	Technology	Version / Notes
CAP Runtime	Node.js with @sap/cds	@sap/cds v7.x
Database (local development)	SQLite	Embedded, no separate installation needed
Database (production)	SAP HANA Cloud	BTP Standard Plan
API Protocol	OData V4	Automatically generated by CAP framework
Authentication	SAP XSUAA	OAuth2 via BTP — xs-security.json
Deployment	Cloud Foundry (BTP)	MTA-based deployment using mta.yaml
CI/CD	GitHub Actions	ci.yml, ci-cd.yml, pr-checks.yml
Frontend	SAP Fiori Elements	Annotations-driven List Report and Dashboard
Unit Testing	Jest + @sap/cds/lib	8 test files, 37 REST test cases
Version Control	Git / GitHub	Feature-branch workflow

4. Step-by-Step Development Guide

Phase 1 — Environment Setup

Step 1: Installing Prerequisites

I began by setting up my local development environment. The following tools were installed before any CAP code was written.

- Node.js LTS v20+ was installed from nodejs.org as the runtime for the CAP framework.

- The SAP CDS command-line tool was installed globally using: npm install -g @sap/cds-dk
- The SAP BTP CLI was installed for managing BTP subaccount resources: npm install -g @sap/btp-cli
- The Cloud Foundry CLI was installed from SAP Tools to enable cf push and deployment commands.
- Visual Studio Code was configured with the SAP CDS Language Support extension for syntax highlighting and code completion.
- A free SAP BTP Trial account was created at account.hanatrial.ondemand.com. SAP HANA Cloud and the Continuous Integration service were enabled within the trial subaccount.

Step 2: Initialising the CAP Project

The project skeleton was generated using the CDS CLI and then extended with the required service bindings.

- cds init novabuild-p2p-analytics — created the project root with the standard db/, srv/, and app/ folders.
- cds add hana --for production — added the HANA adapter to the production profile inside package.json.
- cds add xsuaa — generated the xs-security.json and wired up the XSUAA middleware.
- cds add mta — generated the mta.yaml descriptor needed for BTP Cloud Foundry deployment.
- The completed project structure includes db/, srv/, app/webapp/, test/, scripts/, .github/workflows/, xs-security.json, mta.yaml, and package.json.

Phase 2 — Data Modelling (CDS Schema)

Step 3: Defining Core CDS Entities in db/schema.cds

I defined eight primary entities to cover the full Procure-to-Pay cycle. Each transactional entity uses SAP's managed aspect, which automatically provides createdAt, createdBy, modifiedAt, and modifiedBy fields without any additional code. The namespace is novabuild.p2p.

Entity	Purpose	Key Fields
Vendors	Supplier master data with GST and PAN	vendorID, gstNumber, panNumber, paymentTerms, creditLimit
Materials	Material master with HSN codes and GST rates	materialID, hsnCode, gstRate, valuationClass, standardPrice
PurchaseRequisitions	PR header with composition to PRItems	prNumber, requestedBy, plant, status, totalValue
PRItems	Individual line items within a PR	ID (UUID), pr_prNumber, material_materialID, quantity, unitPrice
PurchaseOrders	PO header with composition to POItems	poNumber, vendor_vendorID, plant, totalValue, taxAmount, status
POItems	PO line items tracking received and invoiced qty	ID (UUID), receivedQty, invoicedQty, gstRate, taxAmount
GoodsReceipts	GR posted against a PO with movement type 101	grNumber, poReference, movementType, postingDate, deliveryNote
InvoiceVerifications	Vendor invoice matched to GR and PO	invoiceNo, baseAmount, taxAmount, invoiceAmount, dueDate, status
AuditLog	Immutable log of all approval and rejection actions	entityType, entityID, action, performedBy, timestamp

Key design decisions in the schema include the use of Composition of many for PR items, PO items, GR items, and invoice items so that the child records are owned by and cascade-deleted with their parent. Associations

are used between PurchaseOrders and Vendors to allow JOIN-free navigation in OData queries. Decimal fields use (15,2) precision throughout to handle INR amounts reliably.

Step 4: Creating Analytical CDS Views in db/views.cds

Six read-only CDS views were built on top of the core entities to support reporting and analytical queries. These views are annotated with @readonly to prevent any write operations through them.

- P2PSummaryView — projects key fields from PurchaseOrders including the vendor, plant, ordered amount, tax, and status. This is the main view used by the list report.
- PRtoPoConversionView — surfaces PR status, requested-by user, requested date, and value so that the conversion rate from requisition to order can be tracked.
- OpenPOTrackerView — applies a WHERE clause to PurchaseOrders excluding CLOSED, CANCELLED, and INVOICED statuses. This gives procurement managers a live list of POs still requiring action.
- GRSummaryView — exposes goods receipt data including movement type 101, posting date, delivery note, and value for each GR.
- InvoiceTrackerView — shows invoice status, due date, base amount, tax, and total invoice amount to support accounts payable monitoring.
- VendorSummaryView — filters to active vendors only and projects the fields needed for vendor performance analysis.

Phase 3 — Service Definition

Step 5: Defining the OData Service in srv/p2p-service.cds

The service is exposed under the path /p2p and is decorated with @requires: 'authenticated-user' at the service level, ensuring that every endpoint is protected by XSUAA before a request reaches any handler.

- All six transactional entities and all six analytical views are exposed through the service. Transactional entities carry @insertonly, @readonly, or full CRUD capabilities depending on their role in the P2P flow.
- A custom OData action approvePR(prNumber: String) returns String is bound to PurchaseRequisitions. It validates the current status, transitions it to APPROVED, and writes an entry to AuditLog.
- A custom OData action rejectPR(prNumber: String, reason: String) returns String is likewise bound to PurchaseRequisitions and writes a REJECTED entry to AuditLog.
- A custom OData function getOpenPOs(plant: String) returns array of PurchaseOrders queries only open orders for a given plant, which is used by the dashboard KPI panel.
- A custom OData function getDashboardKPIs() aggregates five KPIs — open PO count, pending PR count, month-to-date spend, pending invoice count, and overdue deliveries — in a single call for the Fiori dashboard.

Step 6: Implementing Business Logic in srv/p2p-service.js

All server-side business logic is implemented using CAP's event-handler model. The file exports a single function that receives the srv object and registers handlers using srv.before, srv.on, and srv.after.

- srv.before('CREATE', 'PurchaseOrders', ...) — auto-generates the PO number in the format PO followed by seven zero-padded digits, validates that the referenced vendor exists and is active, copies the vendor's payment terms onto the PO header, and sets the default purchasing organisation to NBPO.
- srv.on('approvePR', ...) — reads the current PR status, raises req.error(409, ...) if it is not PENDING, updates the status to APPROVED, and inserts an AuditLog record with the approver's identity taken from req.user.id.
- srv.on('rejectPR', ...) — follows the same guard logic and transitions the PR to REJECTED, also logging the rejection reason.
- srv.after('CREATE', 'GoodsReceipts', ...) — iterates over the GR line items and increments the receivedQty on the matching POItem records. If total received quantity exceeds the ordered quantity, req.error(400, 'Over-delivery not permitted') is raised and the GR is rolled back.
- srv.on('getDashboardKPIs', ...) — runs five SELECT COUNT and SELECT SUM queries against the database and returns the aggregated result as a plain JSON object.

- All validation failures use req.error() rather than thrown exceptions, which produces correctly formatted OData error responses with HTTP status codes 400 or 409.
- cds.log(__filename) is used at the entry and exit of every handler for structured logging that is readable in BTP Application Logging.

Phase 4 — Sample Data Seeding

Step 7: Creating Seed CSV Files in db/data/

Six CSV files were created to populate the database on cds watch (local) and on cds deploy (HANA). The data was designed to exercise all the interesting paths through the service handlers.

- novabuild.p2p-Vendors.csv — 10 vendors including Tata Steel, Siemens India, Bosch India, Larsen and Toubro, Thermax, Cummins, and others. Each record carries a valid GST number, PAN, state, IFSC code, and credit limit.
- novabuild.p2p-Materials.csv — 20 materials covering steel coils, copper wire rod, zinc ingots, PLC units, MRO items, and service materials. Each record has an HSN code and a GST rate (12 or 18 per cent).
- novabuild.p2p-PurchaseOrders.csv — 15 purchase orders in various statuses: OPEN, PARTIALLY_DELIVERED, FULLY_DELIVERED, INVOICED, and CANCELLED. Total spend across all POs exceeds INR 72 lakh.
- novabuild.p2p-GoodsReceipts.csv — Goods receipts are posted for 10 of the 15 POs, intentionally leaving 5 in open or partially-delivered status to test the OpenPOTrackerView.
- novabuild.p2p-InvoiceVerifications.csv — Invoices are raised against 8 GRs. One invoice is deliberately left in PENDING status to ensure the getDashboardKPIs function reports at least one pending invoice.
- novabuild.p2p-PurchaseRequisitions.csv — 10 PRs, 4 in PENDING status and the remainder in APPROVED or REJECTED, providing data for both the approvePR action test and the PRtoPoConversionView.

Step 8: Running and Testing Locally

Local testing was performed using cds watch, which starts an in-memory SQLite instance, seeds it from the CSV files, and serves all OData endpoints on http://localhost:4004.

- GET /p2p/PurchaseOrders — returns all 15 records including expand on items.
- GET /p2p/P2PSummaryView — confirms the analytical view flattens the PO data correctly.
- GET /p2p/OpenPOTrackerView — confirms only OPEN and PARTIALLY_DELIVERED records are returned.
- POST /p2p/PurchaseRequisitions/approvePR with body {prNumber:'PR0000006'} — status transitions to APPROVED and an AuditLog entry is created.
- POST with an already-approved PR — confirms a 409 error is returned.
- POST of a GoodsReceipt with quantity exceeding the PO line — confirms a 400 error is returned and the GR is not persisted.
- A tests.http file containing 37 individual test cases was written for the VS Code REST Client. All cases pass on the local SQLite instance.

Phase 5 — SAP HANA Cloud Integration

Step 9: Deploying the Schema to SAP HANA Cloud

After local testing was complete, the CDS schema was deployed to the HANA Cloud instance on BTP.

- An SAP HANA Cloud instance was created in the BTP trial subaccount using the free 30 GB tier.
- A HANA service instance was created via the CF CLI: cf create-service hana-cloud hana novabuild-hana-db.
- package.json was reviewed to confirm that the [production] profile references the HANA adapter.
- cds deploy --to hana was executed to push the schema. All tables, views, and seed data were created successfully in HANA Cloud. The result was verified in the HANA Database Explorer.

Phase 6 — Security with XSUAA

Step 10: Configuring Authentication and Authorisation

The xs-security.json file generated by cds add xsuaa was extended to define custom scopes and role templates.

- Two scopes were defined: P2P.Read for read-only access to all analytical views and entities, and P2P.Approve for the ability to call the approvePR and rejectPR actions.
- Two role templates were created: P2PViewer (assigned P2P.Read) and P2PApprover (assigned both P2P.Read and P2P.Approve).
- @requires annotations were added per entity and per action in the service CDS file so that the CAP framework enforces authorisation at the handler level.
- The XSUAA service instance was created: cf create-service xsuaa application novabuild-xsuaa -c xs-security.json.
- Two BTP user accounts were used during testing: one assigned only P2PViewer and one assigned P2PApprover. The viewer account received a 403 error when calling approvePR, confirming the authorisation check works correctly.

Phase 7 — MTA Deployment to SAP BTP

Step 11: Building and Deploying the Multi-Target Application

The final deployment was performed using the Multi-Target Application build tool and the CF deploy plugin.

- mta.yaml was reviewed and finalised. It declares three modules — novabuild-p2p-analytics (the CAP Node.js application), novabuild-p2p-analytics-db-deployer (the HDI deployer), and novabuild-p2p-analytics-app (the Fiori frontend) — and two resources: the HANA Cloud service instance and the XSUAA service instance.
- mbt build was executed in the project root to produce the MTAR archive novabuild-p2p_1.0.0.mtar.
- cf login -a https://api.cf.us10.hana.ondemand.com was used to authenticate to the BTP Cloud Foundry environment.
- cf deploy mta_archives/novabuild-p2p_1.0.0.mtar deployed all three modules and bound the service instances. The deployment completed without errors.
- cf apps confirmed all three application instances are in the STARTED state. The live OData service URL was retrieved from cf app novabuild-p2p-analytics.
- All 37 REST test cases from tests.http were re-executed against the production URL using Postman. All cases passed.
- A GitHub Actions CI/CD pipeline defined in .github/workflows/ci-cd.yml automates the build, test, and deploy sequence on every push to the main branch.

Phase 8 — SAP Fiori Elements Frontend

Step 12: Building the Analytics Dashboard

An SAP Fiori Elements-based frontend was developed as an optional enhancement. The dashboard is served from the app/webapp/ folder and communicates with the CAP backend through the OData V4 model.

- CDS UI annotations were added in srv/annotations.cds defining @UI.LineItem for the P2PSummaryView list report and @UI.FieldGroup for the detail page.
- The Dashboard.controller.js file implements a multi-tab analytics view that loads KPI tiles (open POs, pending PRs, month-to-date spend, pending invoices, overdue deliveries), a purchase order list, a pending PR list with Approve and Reject buttons, and a vendor performance ranking. All data is fetched from the OData V4 endpoints on startup.
- The getDashboardKPIs() custom function is called first to populate the five KPI tiles. The remaining entity sets are fetched in parallel using Promise.all for performance.
- The Approve button on the pending PR list calls the approvePR OData action and optimistically removes the PR from the list on success. The Reject button follows the same pattern with rejectPR.
- The frontend falls back to mock data when the OData model is not bound, allowing the UI to be demonstrated standalone in the Fiori sandbox without a live BTP connection.

5. Project Folder Structure

The final project repository is organised as follows. The structure follows the standard SAP CAP project layout and has been extended with scripts, CI/CD workflows, and a dedicated test directory.

File / Folder	Purpose
db/schema.cds	All core entity definitions: Vendors, Materials, PurchaseRequisitions, PRItems, PurchaseOrders, POItems, GoodsReceipts, GRItems, InvoiceVerifications, InvoiceItems, AuditLog
db/views.cds	Six read-only analytical CDS views built on the core entities
db/data/	Six CSV seed files providing 10 vendors, 20 materials, 15 POs, 10 GRs, 10 invoices, and 10 PRs
srv/p2p-service.cds	OData V4 service definition: entity exposures, custom actions (approvePR, rejectPR), custom functions (getOpenPOs, getDashboardKPIs)
srv/p2p-service.js	Business logic: before/on/after event handlers, validation, AuditLog writing
srv/annotations.cds	Fiori UI annotations: LineItem, FieldGroup, Chart, SelectionFields
app/webapp/	SAP Fiori Elements frontend: manifest.json, Component.js, Dashboard.controller.js, index.html, i18n
test/	Eight Jest test files and tests.http with 37 REST cases covering unit, integration, authorisation, and end-to-end P2P flow
scripts/	Shell scripts: deploy-hana.sh, deploy-cf.sh, seed-local.sh
.github/workflows/	GitHub Actions pipelines: ci.yml (lint, test, build), ci-cd.yml (full deploy to BTP), pr-checks.yml
xs-security.json	XSUAA configuration: P2P.Read and P2P.Approve scopes, P2PViewer and P2PApprover role templates
mta.yaml	Multi-Target Application descriptor for BTP Cloud Foundry deployment
package.json	Node.js dependencies, Jest configuration, CDS profiles (development/production), mock user definitions

6. Testing and Validation

6.1 Unit Tests

Unit tests were written using Jest and the @sap/cds/lib test utilities. Each service handler has a corresponding test file. The table below lists the key test cases.

Test ID	Test Description	Expected Result
UT-01	Create a PurchaseOrder with a valid, active vendor	PO is created and auto-number is assigned in PO0000XXX format
UT-02	Create a PurchaseOrder referencing a non-existent vendor	HTTP 400 with message: Vendor not found
UT-03	Call approvePR on a PR currently in PENDING status	PR status changes to APPROVED; AuditLog entry created

UT-04	Call approvePR on a PR that is already in APPROVED status	HTTP 409 with message: PR is already approved
UT-05	Call rejectPR with a rejection reason	PR status changes to REJECTED; AuditLog entry created with reason
UT-06	Post a GoodsReceipt where quantity does not exceed PO line	GR is persisted and POItem.receivedQty is updated
UT-07	Post a GoodsReceipt where quantity exceeds the PO line quantity	HTTP 400 with message: Over-delivery not permitted; GR is rolled back
UT-08	GET P2PSummaryView	Returns flattened PO data including vendorID, orderedAmount, and poStatus
UT-09	GET OpenPOTrackerView	Returns only records with status OPEN or PARTIALLY_DELIVERED
UT-10	Call getOpenPOs(plant='NB10')	Returns only open POs for plant NB10
UT-11	Call getDashboardKPIs()	Returns an object with five numeric KPI fields
UT-12	Unauthenticated request to any service endpoint	HTTP 401 Unauthorized from XSUAA middleware
UT-13	P2PViewer user calls approvePR	HTTP 403 Forbidden — P2P.Approve scope not present

6.2 Integration Tests

An end-to-end integration test in test/p2p-integration.test.js exercises the full P2P chain in a single test run: a PR is created in PENDING status, approvePR is called to transition it, a PO is created referencing the PR, a GR is posted against the PO, and an invoice is raised against the GR. Each step asserts the correct HTTP status code and the correct document status in the database. The same sequence is also covered by tests.http case entries 01 through 08, which were executed against the deployed BTP application using Postman.

6.3 C I / C D Pipeline

A GitHub Actions pipeline defined in .github/workflows/ci-cd.yml runs automatically on every push to the main branch. The pipeline steps are: checkout, npm install, ESLint static analysis, Jest unit tests with coverage reporting, mbt build to produce the MTAR archive, and cf deploy to the BTP Cloud Foundry space. A pull-request check pipeline in pr-checks.yml runs the lint and test steps on every PR before it can be merged.

7. Learning Outcomes and Certification Alignment

Every component of this project was chosen to directly address an examined topic area in the SAP Certified Associate Backend Developer SAP CAP certification. The table below maps each exam topic to the corresponding artefacts produced in this project.

Exam Topic Area	How This Project Addresses It
CDS Data Modelling	Eleven entities defined in db/schema.cds using managed aspect, compositions, associations, and decimal types; six analytical views in db/views.cds with @readonly, @insertonly, and filter predicates
OData V4 Service Exposure	p2p-service.cds exposes all entities and views, defines two bound actions and two unbound functions, and applies @requires annotations at service and entity level

Service Implementation	p2p-service.js registers srv.before, srv.on, and srv.after handlers; implements PO number generation, vendor validation, PR approval and rejection workflow, GR over-delivery check, and KPI aggregation
SAP HANA Cloud Integration	Schema deployed with cds deploy --to hana; HDI deployer module included in mta.yaml; HANA-specific view annotations applied
Authentication and Authorisation	xs-security.json defines P2P.Read and P2P.Approve scopes; @requires annotations enforce scope checks per entity and per action; tested with both P2PViewer and P2PApprover roles
SAP BTP Cloud Foundry Deployment	mta.yaml defines three modules and two resource bindings; MTAR archive built with mbt and deployed with cf deploy; application verified at live BTP URL
Side-by-Side Extension Pattern	Custom CAP service acts as an extension to a hypothetical S/4HANA backend, exposing P2P data through OData V4 without modifying the core SAP system
Testing and Quality Assurance	Eight Jest test files, 37 REST test cases, end-to-end integration test, ESLint static analysis, and GitHub Actions CI/CD pipeline

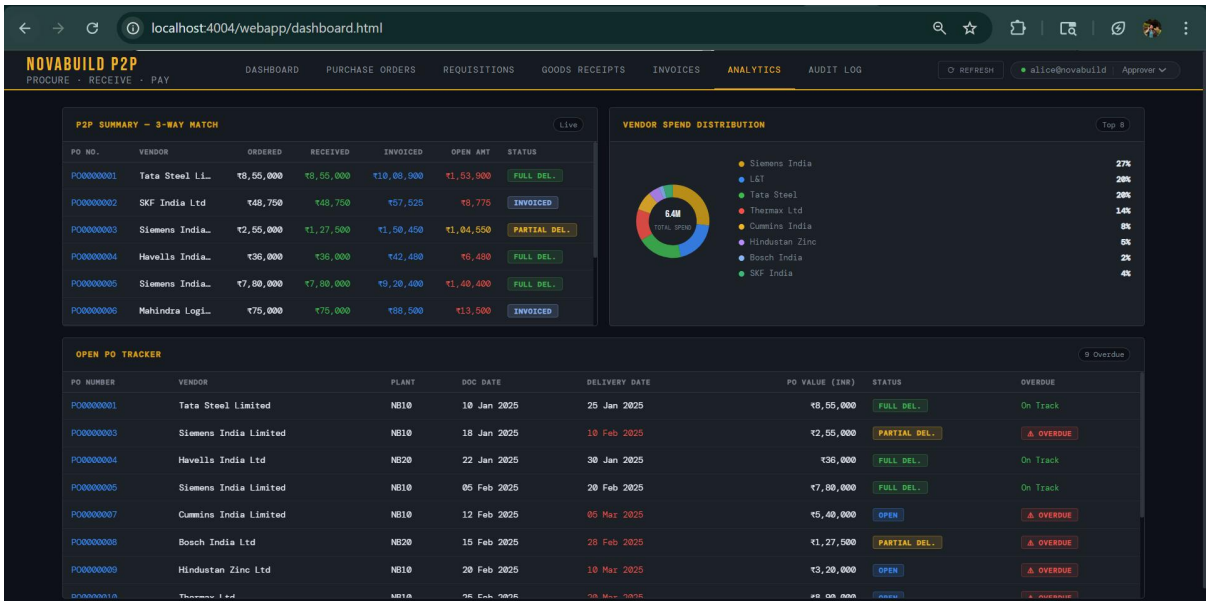
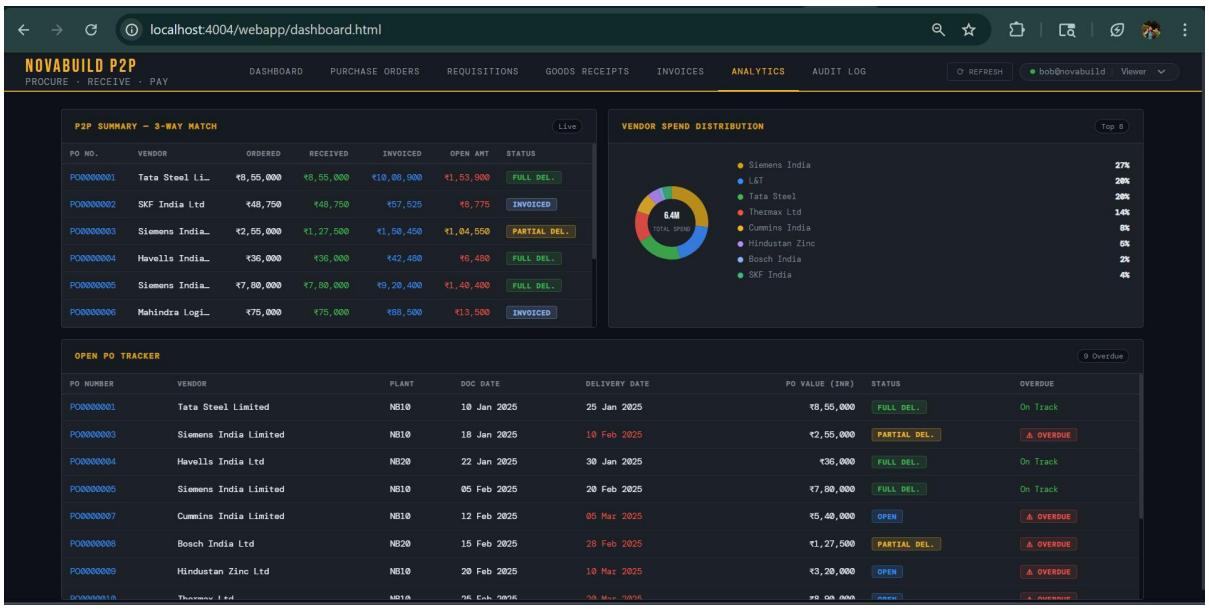
8. References

The following resources were consulted during the development of this project:

- SAP CAP Official Documentation — cap.cloud.sap/docs
- SAP BTP Trial Account — account.hanatrial.ondemand.com
- SAP Learning Journey: SAP CAP Backend Developer — learning.sap.com
- SAP CAP Samples Repository — github.com/SAP-samples/cloud-cap-samples
- SAP HANA Cloud Documentation — help.sap.com/docs/hana-cloud
- OData V4 Specification — odata.org/documentation
- SAP Fiori Elements Documentation — experience.sap.com/fiori-design-web
- Cloud MTA Build Tool Documentation — sap.github.io/cloud-mta-build-tool
- KIIT SAP Project Work Guidance document shared by the faculty
- Capstone Project Submission Guidelines V2 shared by the faculty

9. Screenshots of the dashboards

Approver Dashboard:



Viewer Dashboard: